

CO2 assessment

Name : Kaviya . R

Reg No : 192521075

Course code : CSA0313

Course Name : Data structure.

Linked list configuration

a) operations :

L1 . append (10)

L1 . append (20)

L1 . append (15)

step 1 :

(append (10))

current



[10] → NULL

step 2: (append (20))

current



[10] → NULL

step 3: (append (15))

current



[10] → [20] → NULL

Final linked list

[10] → [20] → [15] → NULL

b) Initially empty

step 1 (append (10))

[10]

step 2: (append (20))

[10] → [20]

step 3: (append (15))

[10] → [20] → [15]

step 4 (move to start () current moves to first node)

current



[10] → [20] → [15]

Step 5 (Insert (39))

Current



[39] → [10] → [20] → [15]

Step 6 (Next ())

Current moves to 10

[39] → Current



[10] → [20] → [15]

Step 7: [Insert (12)]

[39] → Current



[12] → [10] → [20] → [15] → NULL

Final list:

39 → 12 → 10 → 20 → 15 → NULL

2) Binary Search

Given Array:

Index:	0	1	2	3	4	5	6	7	8	9
Value:	66	111	111	323	432	543	643	849	777	888

Case 1:

key = 111
low = 0 high = 9
mid = $(0 + 9) / 2 = 4$
432 > 111
Search left
low = 0 high = 3
mid = 1
111 found.

Position = Index 1 (2nd position)

Case 2:

key = 888
low = 8 high = 9
mid = 4 → 322
Search right
mid = 7 → 649
Search right

Position = Index 9 (10th position)

case : 3

key = 999
432
↓
649
↓
777
↓
888
↓
Not found

Binary search algorithm:

- 1) set low = 0
- 2) set high = 1
- 3) Find mid = $(\text{low} + \text{high}) / 2$
- 4) If key == A[mid]
- 5) Else if key < A[mid]
- 6) Else
- 7) Repeat until low > high
- 8) If low > high

Time Complexity:

- * Best Case = $O(1)$
- * Average = $O(\log n)$
- * Worst = $O(\log n)$

3) Stack (LIFO)

Array size = 5

Initial Top = -1

Operation	stack
Push (45)	45
push (22)	45 22
push (11)	45 22 11
Push (77)	45 22 11 77
Pop ()	45 22 11
push (44)	45 22 11 44
push (33)	45 22 11 44 33

Pop()	45 22 11 44
Pop()	45 22 11
Pop()	45 22

Final stack :

TOP
22
↓
45

Array Representation :

0 1 2 3 4
45 22 - - -

Peak (Top) Element

22
Index

1
Time complexity

push = $O(1)$

Pop = $O(1)$

Peek = $O(1)$

4) expression conversion

expression:

$((a+b) + c * (d+e) + f) * (g+h)$

Prefix :

$* + + + a b * c + d e f + g h$

without spaces :

$* + + + a b * c + d e f + g h$

Postfix :

$a b + d e + * + f + g h + *$

5)

Queue vs priority Queue

Queue	priority Queue.
FIFO Order First come first serve Simple implement Insertion $O(1)$ Deletion $O(1)$ Fair to everyone longer wait for VIP	Priority Based VIP served first more complex Insertion $O(\log n)$ (heap) Deletion $O(\log n)$ less fair short waiting for VIP

Comparison :

Simple Queue :

- *) customer are served in arrival order
- *) Fair to everyone.
- *) easy to implement
- *) constant - time enqueue and deque.
- *) VIP - customer must wait.